Cloud IAM Demystified: AWS Permissions Cheatsheet for Beginners

Identity and Access Management (IAM) is the core gatekeeper of your entire cloud infrastructure. Understanding how users, roles, and policies interact ensures you build secure applications while avoiding costly security breaches.

What you'll learn:

1. IAM Users vs Roles: Who vs What
2. Policies: Crafting Explicit Cloud Permissions
3. Access Keys: Secure Programmatic Access
4. Policy Simulator: Test Access Before Deploying

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

IAM Users vs Roles: Who vs What

IAM Users represent human beings or specific long-term entities with static credentials. IAM Roles are temporary access badges assumed by applications, services, or external users for short tasks. Use roles whenever services talk to each other so you never have to hardcode long-term credentials into source code.

```
$aws iam assume-role --role-arn arn:aws:iam::123456789012:role/DevRole --role-session-name TestSessio
```

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Policies: Crafting Explicit Cloud Permissions

Policies are JSON documents defining allowed or denied actions on specific cloud resources. Everything in AWS is denied by default until an explicit allow rule is attached. Attaching AWS-managed policies via CLI is the fastest way to grant baseline permissions safely.

```
$ aws iam attach-user-policy --user-name john-doe --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnl
```

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Access Keys: Secure Programmatic Access

Access Key IDs and Secret Access Keys let developers run commands from local terminals or automated deployment scripts. Treat access keys with the exact same security level as bank passwords. Never commit them into public repositories and rotate them on a predictable schedule.

```
$aws iam create-access-key --user-name john-doe
```

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Policy Simulator: Test Access Before Deploying

Testing permissions in production leads to unexpected service downtime or accidental security gaps. The AWS IAM Policy Simulator evaluates permissions against specific API calls beforehand. Use it to confirm that actions are properly allowed or blocked before granting real access.

```
$ aws iam simulate-principal-policy --policy-source-arn arn:aws:iam::123456789012:user/john-doe --acti
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- \+ Always enforce Multi-Factor Authentication (MFA) on all user accounts.
- \+ Follow the Principle of Least Privilege: grant only minimum required permissions.
- \+ Use IAM Roles instead of permanent Access Keys for EC2 instances and Lambda functions.
- \+ Organize users into IAM Groups to simplify permission management.
- \+ Rotate Access Keys every 90 days and delete unused active credentials.

Watch Out For

- ! Using the AWS Root account for daily operational CLI tasks.
- ! Hardcoding AWS Access Key ID and Secret Access Key into Git code repositories.
- ! Attaching full AdministratorAccess policy to junior team members for convenience.
- ! Forgetting to revoke IAM credentials immediately when an employee departs.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh